Lesson 1 — Introduction to Distributed Systems

INTE 22253 Distributed Systems and Cloud Computing | Source: Lecture 01 (module overview + "Introduction to Distributed Systems") | Sections marked *[outline]* explain topics the slides only name, using standard textbook theory (Coulouris; Tanenbaum & van Steen).

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Define a distributed system and explain each part of the definition (autonomous nodes, message passing, single coherent view, no global clock).
- Explain the six core characteristics, the six advantages and the six challenges from the slides, with an example of each.
- State the **CAP theorem**, explain why a partition forces a choice between C and A, and say which side a bank and a social feed pick.
- Describe the fundamental models: **architectural** (client-server, peer-to-peer, microservices), **interaction** (synchronous vs asynchronous), **failure** (crash, omission, timing, Byzantine) and **security**.
- Use Google, AWS and banking as examples of distributed systems in practice.

ABOUT THE MODULE (LECTURE 01, SLIDES 2–12)

Assessment: 50% continuous assessment (assignment + mini-project), **50% final exam: 2 hours, closed book**. **Module map:** distributed system models → IPC and synchronization → time and consistency → replication and fault tolerance → distributed file systems and security → cloud computing → virtualization and management → cloud storage → distributed transactions → multi-agent systems → distributed search engines → P2P and blockchain. **Learning outcomes:** LO1 compare architectures and communication models; LO2 synchronization and time ordering; LO3 consistency models and fault-tolerant replication; LO4 IaaS/PaaS/SaaS; LO5 distributed file systems and cloud storage; LO6 distributed transactions, multi-agent systems, P2P, blockchain.

1. What is a distributed system?

DEFINITION (LEARN WORD FOR WORD)

A distributed system is **a collection of independent computers that appears to its users as a single coherent system** — working together, communicating over a network, to achieve a common goal.

Take the definition apart. Each phrase is a separate exam point.

Phrase / slide point	What it means for a beginner
Independent (autonomous) computers, "nodes"	Each machine has its own CPU, its own memory and its own operating system. A **node** is just one such computer in the system. Nodes can start, stop or crash independently of each other.
Do not share physical memory	Node A cannot read a variable in Node B's RAM. This is the big difference from a multi-core PC, where all cores share one RAM.
Coordinate solely by passing messages	Because there is no shared memory, the *only* way to cooperate is to send **messages** (packets of data) over the network: "please store this", "here is the result", "are you alive?".
Appears as a single coherent system	The user sends *one* request (e.g. a Google search) and gets *one* answer. They do not see, and should not need to know, that thousands of machines worked on it. Hiding this is called **transparency**.
No global clock	Every node has its own clock and the clocks drift apart. There is no single "true time" all nodes can read, so deciding *which event happened first* is hard.
No shared state — coordination must be explicit	No node automatically knows what the others are doing. If they must agree (e.g. on a bank balance), a protocol must make them agree, using messages.

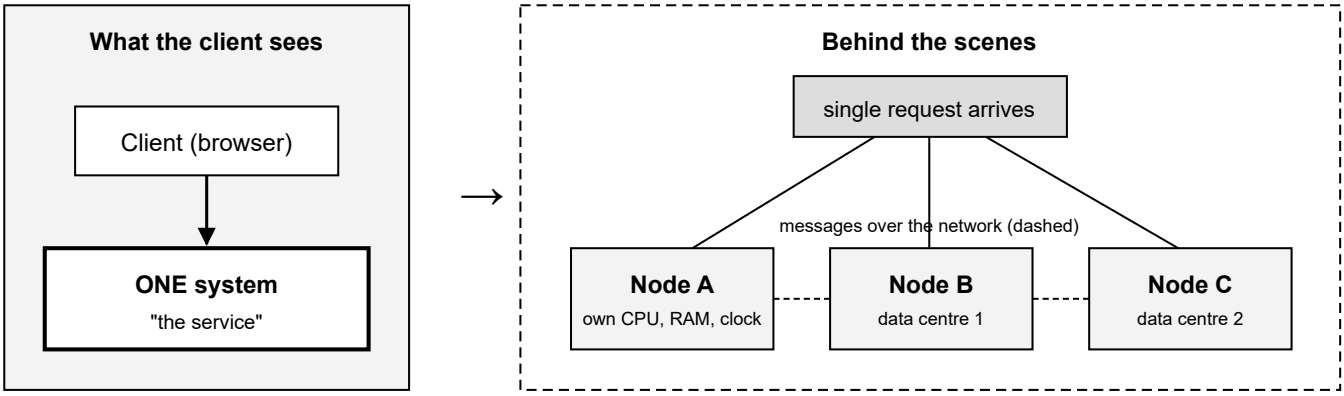


Figure 1 — Slide 16: the client sends one request to "one system"; behind the scenes independent nodes, possibly in different data centres, coordinate by messages.

Distributed vs centralized vs parallel (common comparison question)

Type	Memory	Clock	How parts cooperate	Failure	Example
Centralized	One computer	One clock	Inside one OS	All-or-nothing: the machine works or it is down	A single office server
Parallel (multi-core)	Shared RAM	One (shared)	Shared variables	Usually all-or-nothing	A multi-core laptop
Distributed	**Each node its own**	**Many, drifting**	**Messages over a network**	**Partial: some nodes fail, others keep running**	Google Search, AWS, ATM network

Middleware (named in Lecture 04's recap): the software layer that sits between the applications and the operating systems/network of each node. It hides the differences between machines (heterogeneity) and gives programmers one uniform interface, e.g. "call a function on another machine as if it were local". Examples: RPC frameworks, message queues, web-service platforms.

2. Core characteristics (slide 17)

These six properties are shared by every distributed system. Learn them as a list of six, each with one sentence and one example.

Characteristic	Slide meaning	Explanation and example
Resource sharing	Hardware, data and services are shared across nodes rather than duplicated on one machine.	A **resource** is anything useful: a printer, a disk, a database, a service. Example: all staff use one shared file server; millions of users share Google's index.
Scalability	Capacity grows by **adding more nodes** (horizontal scaling) rather than upgrading one machine.	A system is **scalable** if it keeps working well when users, data or machines grow. Adding machines = **scale out / horizontal**; buying a bigger machine = **scale up / vertical** (Figure 2).
Concurrency	Many nodes process requests simultaneously, independently of one another.	Thousands of users act at the same time, so two requests may touch the same data at once (two people booking the last seat). The system must keep shared data correct.
Fault tolerance	The system keeps running correctly even when individual nodes fail or disconnect.	With many machines, something is always broken. Copies of data (**replication**) and spare nodes let the service continue.
No global clock	Nodes can't rely on perfectly synchronized time, complicating event ordering.	If Node A's clock says 10:00:01 and Node B's says 10:00:00, which event was really first? Clocks can be synchronized only approximately.
Transparency	Distribution is hidden — users interact with one logical system.	The user does not know where data is, how many copies exist, or that a server just failed. Types below.

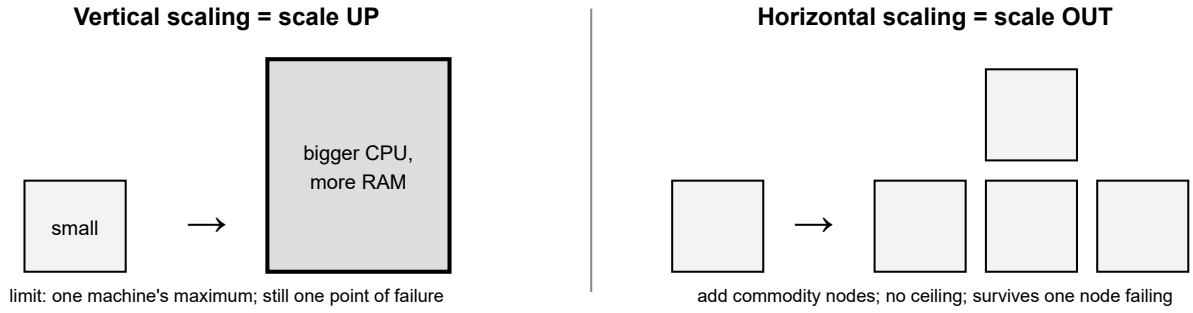


Figure 2 — Scale up vs scale out. Distributed systems scale out.

Forms of transparency (Lecture 04 recap lists six)

Transparency	What is hidden from the user	Example
Access	Differences in how data is represented and how a resource is accessed (local vs remote look the same)	A network drive opens like a local folder
Location	Where a resource physically is	A URL `www.example.com` says nothing about which data centre
Migration	That a resource may move to another location	A VM moved to another host while users keep working
Replication	That several copies of a resource exist	You see one Gmail inbox, stored as several copies
Concurrency	That other users share the same resource at the same time	Two people edit a shared document without corrupting it
Failure	The failure and recovery of a component	A server crashes; your request is silently retried on another

Two more design goals named in the outline *[outline]*

- **Openness**: the system is built on **published, standard interfaces and protocols** (e.g. HTTP, TCP/IP, REST APIs), so new components from different vendors can be added or replaced. An open system is extensible and interoperable.

- **Heterogeneity**: the nodes differ — different hardware, operating systems, networks, programming languages. Middleware and standard protocols hide these differences.
- **Reliability**: the system continues to give correct service over time; closely tied to fault tolerance and availability.

3. Why go distributed? Advantages (slide 18)

Advantage	Explanation
Scalability	Add more machines on demand to handle growth — no ceiling imposed by a single server's hardware.
Fault tolerance	If one node fails, others take over. There is **no single point of failure (SPOF)** for the whole system. A SPOF is one component whose failure stops everything.
Performance	Workloads run **in parallel** across many nodes, cutting response time for large or complex tasks (e.g. MapReduce, section 7).
Resource efficiency	Hardware, storage and bandwidth are pooled and shared rather than sitting idle on one machine.
Geographic reach	Nodes placed near users worldwide reduce **latency** (the delay before a response arrives) and keep services available regionally.
Cost-effective growth	Scaling out with cheap **commodity servers** (ordinary, mass-produced machines) is often cheaper than scaling up one very powerful machine.

4. The hard trade-offs: challenges (slide 19)

Challenge	Why it is hard
Complexity	More moving parts ⇒ harder design, debugging, deployment and testing. A bug may appear only when messages arrive in an unusual order.
Partial failures	Some nodes fail while others keep running. A single PC either works or crashes; a distributed system can be "half working", and a node often cannot tell whether another node crashed or is just slow.
Consistency	Keeping copies of data in sync across nodes is hard, especially under concurrent updates (Lesson 4).
Synchronization	No global clock ⇒ ordering events and coordinating actions is genuinely difficult.
Security	More network communication paths ⇒ a larger **attack surface** (more places an attacker can try to get in).
Network reliability	Latency, **partitions** (the network splits so groups of nodes cannot reach each other) and message loss are normal — they must be designed for, not assumed away.

EXTRA: THE CLASSIC LIST OF CHALLENGES (COULOURIS) — MATCHES THE COURSE OVERVIEW

Heterogeneity · Openness · Security · Scalability · Failure handling · Concurrency · Transparency · Quality of service. If a question says "discuss the challenges of designing a distributed system", you can combine this list with slide 19.

5. The CAP theorem (slide 20)

STATEMENT

In a distributed system you can only **fully guarantee two** of these three properties at the same time: **C**onsistency — every node sees the same data at the same time; **A**vailability — every request gets a response, even if some nodes are down; **P**artition tolerance — the system keeps working despite network failures between nodes.

Why only two? Partitions *will* happen in any real network (cables are cut, switches fail). So P is not optional. When a partition happens, a node that cannot reach the others has only two choices when a request arrives:

- **Refuse or wait** (return an error) ⇒ it stays **consistent** but is **not available** ⇒ a **CP** system.
- **Answer anyway** with the data it has ⇒ it stays **available** but may return **stale** (out-of-date) data ⇒ an **AP** system.

Worked example: one partition, two choices

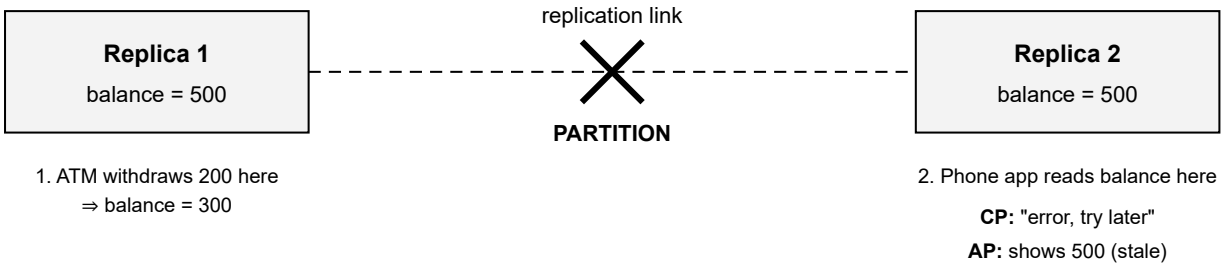


Figure 3 — The link between two replicas breaks. Replica 2 cannot know about the withdrawal.

Step	CP choice (consistency first)	AP choice (availability first)
Withdraw 200 at Replica 1	Allowed only if it can also update Replica 2 (it can't) ⇒ may be refused	Accepted; Replica 1 now 300
Read at Replica 2	Refused / waits until the link is back	Answers 500 — wrong but fast
After the link is repaired	Nothing to fix; data never diverged	Replicas exchange updates and **converge** to 300 (eventual consistency)
Who picks this	**Banking systems** (consistency is non-negotiable)	**Social feeds** (a slightly old feed is acceptable; being down is not)

The **CA** combination is only possible when there is no network to partition (e.g. one database on one site). Lecture 04 extends this (CP/AP examples, PACELC).

6. Fundamental models for distributed systems *[outline]*

Lecture 01 slide 4 names five model topics. A **model** is a simplified description that captures the important properties so we can reason about a design. The slides do not explain them, so here they are at textbook level.

6.1 Architectural models: how the parts are arranged

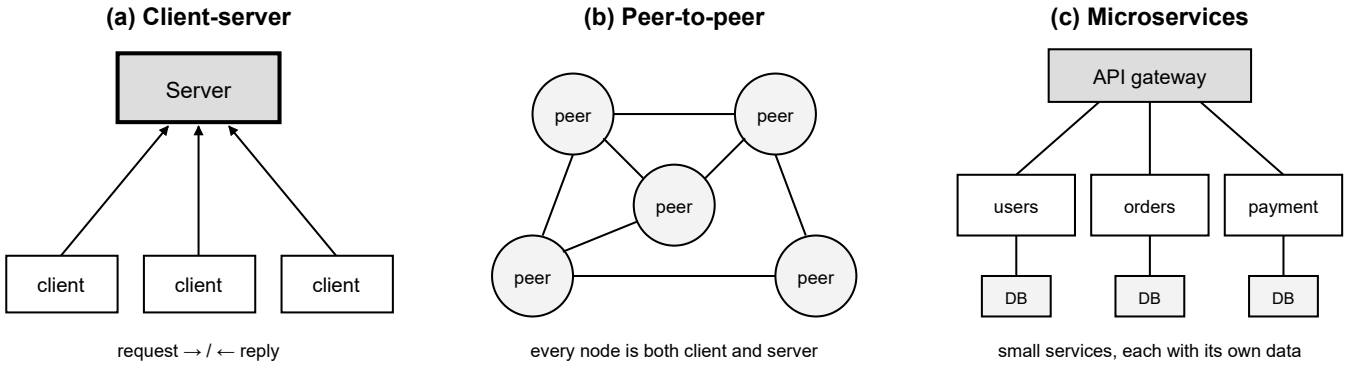


Figure 4 — The three architectural models named on slide 4.

Aspect	Client-server	Peer-to-peer (P2P)	Microservices
Idea	Servers provide services; clients request them (request–reply)	All nodes (peers) are equal and both use and provide resources	One application split into many small, independently deployable services that talk over the network (usually HTTP/REST or messages)
Control	Centralized at the server	Decentralized	Each service owned by one team
Scalability	Limited by the server; add servers to scale	Grows naturally: every new peer adds capacity	Scale each service separately (only "orders" during a sale)
Weak point	Server is a bottleneck and SPOF	Hard to manage, search, secure; peers join and leave (**churn**)	Many network calls; complex deployment and monitoring
Examples	Web browser + web server, email	BitTorrent, blockchain networks	Netflix, Amazon (cloud-native apps, Lesson 2)

6.2 Interaction models: synchronous vs asynchronous

Two meanings are used; know both.

(a) Synchronous vs asynchronous *communication*

- **Synchronous (blocking)**: the sender sends a request and **waits** until the reply arrives before continuing. Simple to program (like a phone call). Example: an HTTP request, RPC.
- **Asynchronous (non-blocking)**: the sender sends the message and **continues immediately**; the reply (if any) is handled later. Example: email, message queues. Better throughput and resilience; harder to program.

(b) Synchronous vs asynchronous *system model*

- **Synchronous system**: known **upper bounds** on process execution time, message delay and clock drift. A timeout can then *prove* a node has crashed.
- **Asynchronous system**: **no bounds** — a message may take any time. You cannot tell a crashed node from a slow one. The Internet behaves like this.

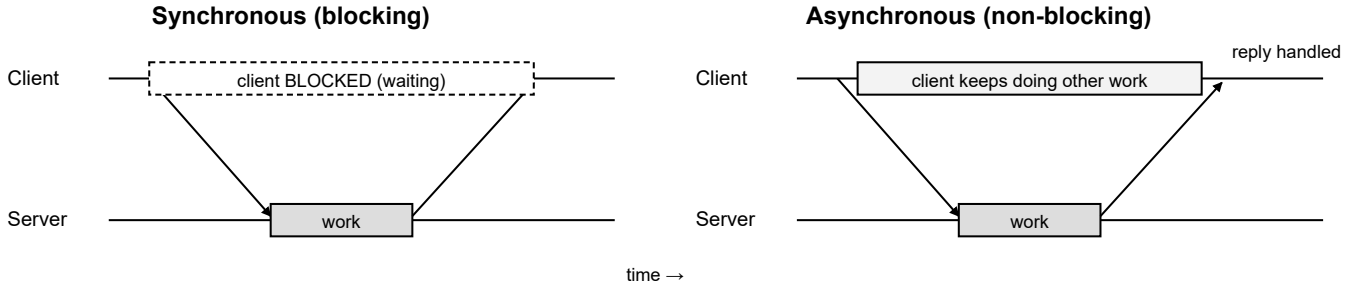


Figure 5 — Synchronous vs asynchronous communication on a time line.

6.3 Failure models: what can go wrong (detailed in Lesson 4)

Failure	What happens	Example
Crash	A process stops and does nothing more; it worked correctly until it stopped	Server loses power
Omission	A process or channel fails to send or receive a message	A request packet is dropped
Timing	The response is correct but arrives outside the allowed time interval (too late)	A video frame that arrives after it was due
Byzantine (arbitrary)	A process behaves in any way at all, including sending wrong or malicious data; the hardest to tolerate	A hacked node sending fake results

Severity increases down the table: a crash is easy to reason about; a Byzantine node may lie.

6.4 Security models: threats and trust

The security model assumes an **enemy (attacker)** who can read, copy, change, insert or replay messages on the network. Main threats (Lecture 01 slide 6 lists them for DFS):

- **Eavesdropping**: reading private messages. **Man-in-the-middle (MITM)**: secretly sitting between two parties and relaying/altering messages.
- **Masquerading / spoofing**: pretending to be another user or server. **Tampering**: changing messages in transit.
- **Replay**: recording a valid message and sending it again later (e.g. replaying "pay Rs. 1000").
- **Denial of Service (DoS)**: flooding a service with requests so genuine users cannot use it.

Defences: **authentication** (prove who you are), **authorization** (decide what you may do), **secure channels** with encryption such as **TLS/SSL**. A **trust hierarchy** means deciding which components are trusted (e.g. a certificate authority, an authentication server) and building other trust on them.

7. Real-world examples (slides 21–23)

7.1 Google: search at planetary scale

- **Massive server farms**: millions of commodity servers across data centres working as one logical system.
- **Distributed file system (GFS)**: files split into **chunks**, replicated across many machines for durability and speed (details: Lesson 7).
- **MapReduce**: huge computations split into small tasks, run in parallel, then combined — built for failure-prone clusters (a failed task is simply re-run on another machine).

Why it matters: a query answered in milliseconds is the product of thousands of machines cooperating.

Worked example: MapReduce word count

Count words in two documents using two "map" workers and one "reduce" step.

Step	Worker 1 (doc: "cloud data cloud")	Worker 2 (doc: "data node")
Map: emit (word, 1) for every word	(cloud,1) (data,1) (cloud,1)	(data,1) (node,1)
Shuffle: group pairs by word	cloud → [1,1] data → [1,1] node → [1]	
Reduce: add the list for each word	**cloud = 2, data = 2, node = 1**	

The map steps run at the same time on different machines (performance). If Worker 2 crashes, the master re-runs its map task elsewhere (fault tolerance).

7.2 AWS: cloud infrastructure as a service

- **Global regions and availability zones**: data centres are grouped into **regions** (geographic areas) and **availability zones** (separate facilities with independent power/cooling inside a region), so apps can survive losing an entire facility.
- **Elastic Compute (EC2)**: virtual servers spin up or down automatically as demand changes.
- **Distributed storage (S3)**: objects replicated across multiple devices and facilities for extreme durability.

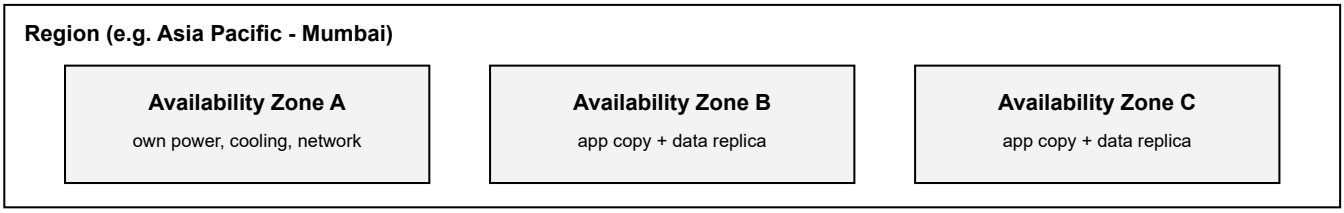


Figure 6 — Regions contain several isolated availability zones; copies in different zones survive a facility failure.

7.3 Banking systems: consistency at stake

- **Strong consistency**: a withdrawal at an ATM must be reflected instantly everywhere — no two systems may show different balances.

- **Distributed transactions**: transfers between accounts at different banks use protocols ensuring **both sides commit, or neither does** (atomicity; Two-Phase Commit, Lesson 4).
- **High availability**: core banking and card networks must stay up nationally even if individual data centres fail.

Banking shows the CAP trade-off in action: consistency is non-negotiable, even at some cost to availability.

8. Key takeaways (slide 24)

1. **Many machines, one experience**: independent nodes cooperate over a network to appear as a single unified system.
2. **Power comes with complexity**: scalability and fault tolerance are real wins, but they demand careful design for failure, consistency and timing.
3. **Trade-offs are the design**: CAP reminds us that C, A and P cannot all be maximized at once.
4. **It's already everywhere**: Google, AWS and banking run daily life on distributed systems.

9. Exam-style questions with model answers

Q1. Define a distributed system and explain three of its characteristics. (6 marks)
A distributed system is a collection of independent computers that appears to its users as a single coherent system, communicating over a network to achieve a common goal. (1) *Resource sharing*: hardware, data and services are shared across nodes, e.g. a shared file server. (2) *Concurrency*: many nodes process requests simultaneously, so shared data must be protected from conflicting updates. (3) *No global clock*: each node has its own clock, so ordering events requires explicit coordination. (Others: scalability, fault tolerance, transparency.)

Q2. Compare horizontal and vertical scaling. Why do distributed systems prefer horizontal scaling? (4)
Vertical scaling (scale up) adds CPU/RAM to one machine; horizontal scaling (scale out) adds more machines. Horizontal scaling is preferred because there is no ceiling set by one machine's hardware, commodity servers are cheaper, and with many nodes there is no single point of failure.

Q3. State the CAP theorem. Explain with an example why a banking system and a social media feed make different choices. (6)
A distributed system can guarantee at most two of Consistency, Availability and Partition tolerance at once. Since partitions always happen, the real choice during a partition is C or A. A bank chooses C (CP): if replicas cannot communicate it refuses or delays requests rather than show a wrong balance, because an incorrect balance causes financial loss. A social feed chooses A (AP): it keeps answering with possibly stale posts, because a slightly old feed is acceptable but being unavailable loses users.

Q4. What is meant by "partial failure"? Why does it make distributed systems harder than centralized ones? (4)
A partial failure is when some components fail while others continue. In a centralized system failure is all-or-nothing. In a distributed system a node cannot easily tell whether another node crashed, is slow, or the network dropped the message, so it must use timeouts, retries and replication, and must avoid leaving work half-done.

Q5. Explain four types of transparency. (4)
Location (user does not know where a resource is), replication (does not know there are several copies), failure (does not notice a component failing and recovering), concurrency (does not notice other users sharing the resource). Also access and migration.

Q6. Compare the client-server and peer-to-peer architectures. (6)
Client-server: dedicated servers provide services and clients request them; control is centralized, simple to manage and secure, but the server is a bottleneck and single point of failure. P2P: every peer acts as both client and server; it scales naturally because each peer adds capacity and has no central SPOF, but it is harder to manage, search and secure, and must cope with peers joining and leaving.

Q7. Distinguish synchronous and asynchronous communication. Give one advantage of each. (4)
Synchronous: the sender blocks until the reply arrives — simple to program and reason about. Asynchronous: the sender continues immediately and handles the reply later — better throughput, and the sender is not stuck if the receiver is slow.

Q8. Describe four failure models. Which is hardest to tolerate and why? (5)
Crash (process stops), omission (message not sent/received), timing (correct response outside the time limit), Byzantine (arbitrary, possibly malicious behaviour). Byzantine is hardest because a faulty node may send different wrong answers to different nodes, so it cannot be detected by silence alone; the others must cross-check and vote.

Q9. Using Google, AWS or banking as an example, explain how a distributed system achieves fault tolerance and performance. (6)
Google: GFS splits files into chunks and replicates each chunk on several machines, so losing one machine loses no data (fault tolerance); MapReduce splits a job into many small tasks run in parallel on thousands of commodity servers (performance), re-running any task whose machine fails.

Q10. List three advantages and three challenges of distributed systems. (6)
Advantages: scalability, fault tolerance (no SPOF), performance via parallelism (also resource efficiency, geographic reach, cost-effective growth). Challenges: complexity, partial failures, consistency of replicated data (also synchronization without a global clock, security attack surface, network unreliability).

10. Common mistakes

Mistake	Correct idea
"A distributed system is many computers sharing memory."	Nodes do **not** share physical memory; they cooperate only by messages.
"CAP: just pick any two you like, e.g. CA for big systems."	Partitions always happen in a real network, so P is required; the real choice is C *or* A during a partition. CA only fits a single site.
Confusing consistency (CAP) with ACID consistency	CAP consistency = all nodes see the same latest data. ACID consistency = a transaction keeps the database rules valid.
"Availability means the system never fails."	It means every request gets a (non-error) response, possibly with old data.
Scale up = adding machines	Scale **up** = bigger machine (vertical); scale **out** = more machines (horizontal).
"No global clock" means nodes have no clocks	Every node has a clock; they are not perfectly synchronized, so there is no single shared time.
Writing "transparency = security" or "= visibility"	Transparency = **hiding** distribution from the user.
Thinking P2P has servers	Every peer is both client and server.

Cheat sheet

Definition & characteristics	
Independent computers that **appear as one coherent system**; no shared memory; message passing; no global clock; no shared state.	
6 characteristics	Resource sharing, Scalability, Concurrency, Fault tolerance, No global clock, Transparency
6 advantages	Scalability, Fault tolerance, Performance, Resource efficiency, Geographic reach, Cost-effective growth
6 challenges	Complexity, Partial failures, Consistency, Synchronization, Security, Network reliability
Transparency	Access, Location, Migration, Replication, Concurrency, Failure
Also	Openness (standard interfaces), Heterogeneity (different HW/OS/lang), Middleware hides it
Scaling	Up = vertical = bigger box; Out = horizontal = more boxes (preferred)

Real-world	
Google	Server farms, GFS (chunks, replicated), MapReduce (map → shuffle → reduce)
AWS	Regions + AZs, EC2 elastic compute, S3 replicated storage
Banks	Strong consistency, distributed transactions (both commit or neither), high availability

CAP theorem	
C	Every node sees the same data at the same time
A	Every request gets a response, even if nodes are down
P	Keeps working despite network failures between nodes
Rule	Only 2 of 3. Partitions happen ⇒ choose C or A
CP	Refuse during partition — banking
AP	Answer with stale data, converge later — social feed

Fundamental models	
Architecture	Client-server (central, SPOF) · P2P (equal peers, churn) · Microservices (small independent services)
Interaction	Sync = block & wait (bounds known) · Async = continue (no bounds)
Failure	Crash < Omission < Timing < Byzantine (arbitrary/malicious)
Security	Eavesdrop, MITM, spoof, tamper, replay, DoS; defend with authentication, authorization, TLS

Exam
Final 50%, 2 hours, closed book.